SINGLE LAYER NEURAL NETWORK

Prof. Dr. Murat EKİNCİ

440638 – Mohamed HASSAN

Department of Computer Engineering

Artificial Neural Systems (BIL5050)

Contents

1. Introduction 1
1.1. System Overview 1
1.2. Neural Network Classifier Interface Components 1
2. Implementation Details 3
2.1. Development Environment and Tools and libraries 3
2.2. System Architecture 3
2.2.1. Class Hierarchy 4
2.2.2. Data Flow Architecture 4
2.3. Core Components 5
2.3.1 Activation Functions 5
2.3.2. Loss Functions 7
2.3.3. Softmax Function 10
2.4. Optimization Methods 11
2.5. Data Processing Pipeline 12
2.5.1. Data Collection 12
2.5.2. Data Preprocessing 13
2.6. Neural Network Implementation 14
2.6.1. Network class Initialization 14
2.6.2. Network Method Initialization 16
2.6.3. Forward Propagation 17
2.6.4. Backward Propagation 19
2.6.5. Training Process 21
2.6.6. Prediction 25
2.6.7. Loss Computation 26
3. Results and Analysis 27
3.1. Single Neuron Results 27
3.1.1. Training Performance 27
3.1.2. Decision Boundaries 28
3.1.3. Performance Analysis 29
3.2. Multi Neuron Results 29
3.2.1. Training Performance 29
3.2.2. Decision Boundaries 29
3.2.3. Performance Analysis 30
4. Conclusions 30

Figures Contents

Figure 1. User interface.....1
Figure 2. Class diagram. 4
Figure 3. Data flow Diagram 4
Figure 4. Decision Boundaries of two classes classification 28
Figure 5. Network Architecture of two classes classification.....28
Figure 6. Decision Boundaries of Multiple classes classification 29
Figure 7. Network Architecture of Multiple classes classification 30

Tables Contents

Table 1. Neural Network components 2
Table 2. Tools and Libraries.....3

1. Introduction

This report presents the implementation and analysis of Single Layer Neural Networks for classification problems in 2-dimensional feature space. We focus on two main architectures:

1.Single Neuron for binary classification

2.Multiple Neurons for multi-class classification

1.1. System Overview

The system provides an interactive tool for implementing and visualizing single-layer neural networks. It features real-time training visualization, flexible parameter adjustment, and support for both binary and multi-class classification in 2D feature space. The interface allows users to input training data through mouse clicks, adjust network parameters dynamically, and observe the evolution of decision boundaries during training.

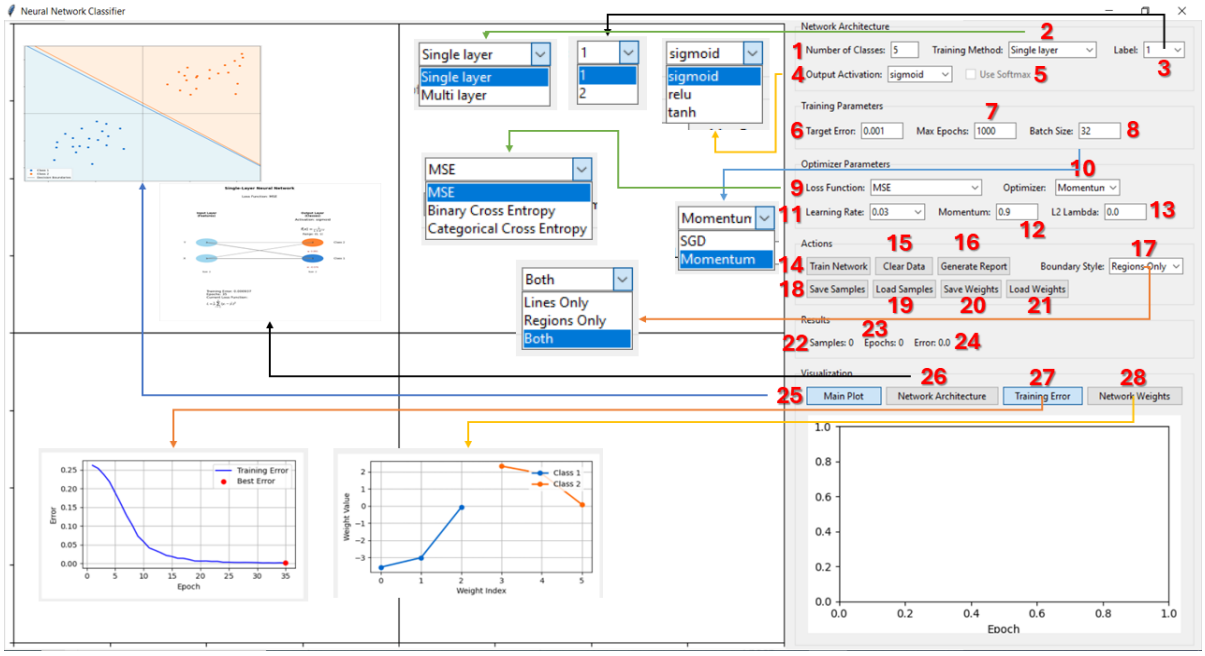

Figure 1. User interface

1.2. Neural Network Classifier Interface Components

The interface consists of four main sections that provide comprehensive control over network architecture, training parameters, optimization settings, and visualization options:

Table 1. Neural Network components

	Section	Component Name	Description	Default/Options
1	**Network Architecture**	Number of Classes	Specifies the number of output classes	Default: 5
2	**Network Architecture**	Training Method	Selects network architecture type	Options: Single Layer, Multi-Layer
3	**Network Architecture**	Label	Selects class label for training	Range: 1 to Number of Classes
4	**Network Architecture**	Output Activation	Defines the activation function for output neurons	Options: sigmoid, relu, tanh
5	**Network Architecture**	Use Softmax	Enables softmax activation for the output layer	Checkbox
6	**Training Parameters**	Target Error	Minimum error threshold for training	Default: 0.001
7	**Training Parameters**	Max Epochs	Maximum number of training iterations	Default: 1000
8	**Training Parameters**	Batch Size	Number of samples per training batch	Default: 32
9	**Optimizer Parameters**	Loss Function	Error calculation method	Options: MSE, Binary Cross Entropy, Categorical Cross Entropy
10	**Optimizer Parameters**	Section Header	Header for optimizer settings	-
11	**Optimizer Parameters**	Learning Rate	Step size for weight updates	Default: 0.03
12	**Optimizer Parameters**	Momentum	Momentum coefficient for optimization	Default: 0.9
13	**Optimizer Parameters**	L2 Lambda	L2 regularization parameter	Default: 0.0
14	**Actions**	Train Network	Button to start the training process	-
15	**Actions**	Clear Data	Button to reset all data	-
16	**Actions**	Generate Report	Button to create an analysis report	-
17	**Actions**	Boundary Style	Visualization style selector	Options: Both, Lines Only, Regions Only
18	**Actions**	Save Samples	Button to save training data	-
19	**Actions**	Load Samples	Button to load training data	-
20	**Actions**	Save Weights	Button to save network weights	-
21	**Actions**	Load Weights	Button to load network weights	-
22	**Results**	Samples	Displays the number of training samples	Default: 0

Table 1 continue				
23	**Results**	Epochs	Displays the current training epoch	Default: 0
24	**Results**	Error	Displays the current error value	Default: 0.0
25	**Visualization**	Main Plot	Tab for data and decision boundary display	-
26	**Visualization**	Network Architecture	Tab for network structure visualization	-
27	**Visualization**	Training Error	Tab for error progression graph	-
28	**Visualization**	Network Weights	Tab for weight visualization	-

2. Implementation Details

This section outlines the technical aspects of the Single Layer Neural Network implementation.

2.1. Development Environment and Tools and libraries

Table 2. Tools and Libraries

Category	Library/Tool	Version	Purpose
Development Environment	Python	3.6+	Primary programming language
GUI Framework	Tkinter	Built-in	Graphical user interface development
Scientific Computing	NumPy	Latest	Numerical computations and array operations
Visualization	Matplotlib	3.x+	Data visualization and plotting
Image Processing	PIL (Pillow)	6.0+	Image handling and processing
Document Generation	ReportLab	3.0+	PDF report generation
Type System	Typing	Built-in	Type hints and annotations
Error Handling	Warnings	Built-in	Warning system and error handling
Date/Time	Datetime	Built-in	Date and time operations
File Operations	IO	Built-in	Input/output operations

2.2. System Architecture

The implementation follows an object-oriented architecture with clear separation of concerns.

2.2.1. Class Hierarchy

The implementation follows an object-oriented design:

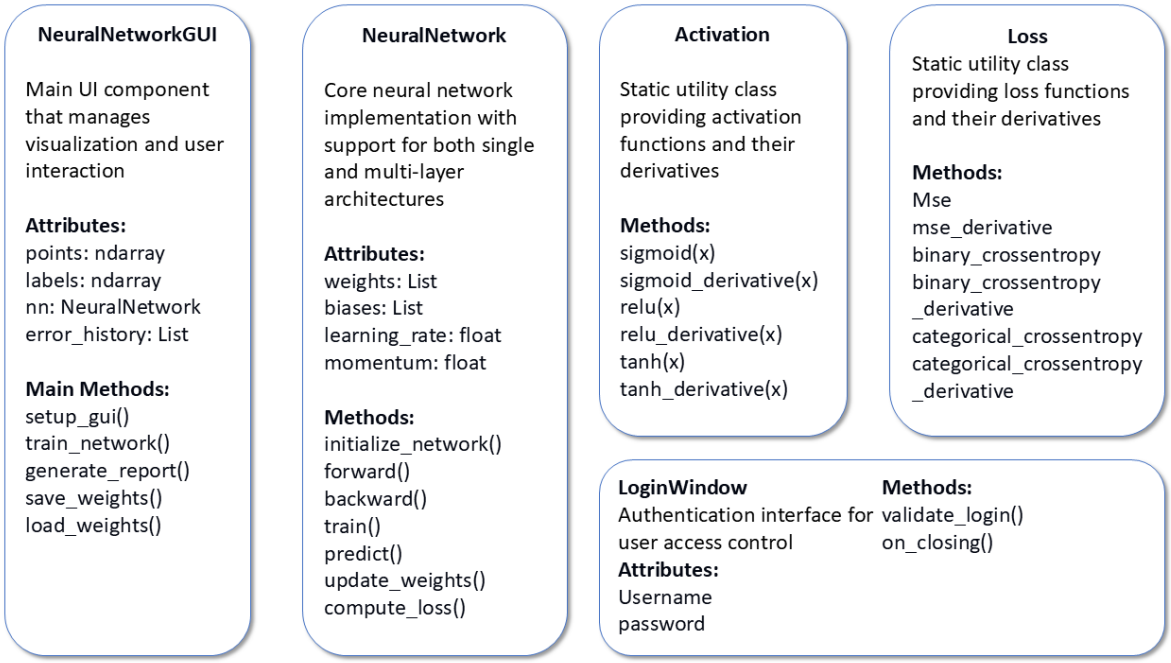

Figure 2. Class diagram.

2.2.2. Data Flow Architecture

The data flow diagram for Sigle layer network

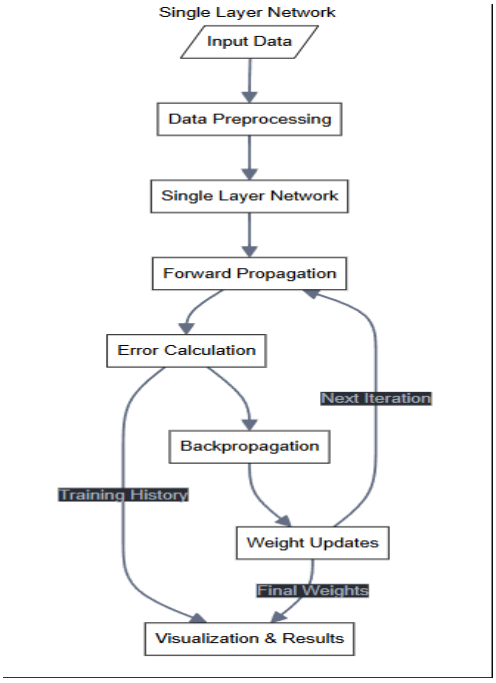

Figure 3. Data flow Diagram

2.3. Core Components

The Single Layer Neural Network relies on the following key elements to enable efficient training and accurate classification.

2.3.1 Activation Functions

Activation functions introduce non-linearity, enabling the neural network to model complex relationships. Each function includes its derivative, essential for backpropagation during training.

a) Sigmoid Function

```
60 class Activation:
61     """Class containing static methods for activation functions and their derivatives."""
62     @staticmethod
63     def sigmoid(x: np.ndarray) -> np.ndarray:
64         """Sigmoid activation function."""
65         return 1.0 / (1.0 + np.exp(-np.clip(x, -500, 500)))
66     @staticmethod
67     def sigmoid_derivative(x: np.ndarray) -> np.ndarray:
68         """Derivative of sigmoid activation."""
69         sig = Activation.sigmoid(x)
70         return sig * (1 - sig)
```

Mathematical Representation: The Sigmoid activation function is defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{1}$$

This implementation, in line **65**, includes the use of `np.clip` to prevent numerical overflow by constraining **x** values to the range [−500,500], ensuring computational stability.

Derivative: The derivative of the Sigmoid function, implemented in line **70**, is computed as:

$$\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x)) \tag{2}$$

It calculates the gradient needed for backpropagation by first evaluating **σ(x)**and then deriving the slope. Both methods take NumPy arrays as input and return NumPy arrays.

b) ReLU Function

```
74  @staticmethod
75  def relu(x: np.ndarray) -> np.ndarray:
76      """ReLU activation function."""
77      return np.maximum(0, x)
78
79  @staticmethod
80  def relu_derivative(x: np.ndarray) -> np.ndarray:
81      """Derivative of ReLU activation."""
82      return np.where(x > 0, 1, 0)
```

Mathematical Representation: The ReLU (Rectified Linear Unit) activation function, implemented in **line 77**, introduces non-linearity by setting negative inputs to 0. It is mathematically defined as:

$$f(x) = max(0, x) \tag{3}$$

This efficient vectorized operation uses **np.maximum()** to process arrays.

Derivative: The derivative of the ReLU function, implemented **in line** 82, is expressed as:

$$f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases} \tag{4}$$

Although technically undefined at x=0x = 0x=0, it is conventionally set to 000. The implementation uses **np.where()** for fast, vectorized condition evaluation.

c) Tanh function

```
84  @staticmethod
85  def tanh(x: np.ndarray) -> np.ndarray:
86      """Tanh activation function."""
87      return np.tanh(x)
88
89  @staticmethod
90  def tanh_derivative(x: np.ndarray) -> np.ndarray:
91      """Derivative of tanh activation."""
92      return 1 - np.tanh(x) ** 2
```

Mathematical Representation: The Tanh (Hyperbolic Tangent) activation function, implemented in **line 87**, maps inputs to a range of (−1,1), offering zero-centered outputs for balanced gradients. It is mathematically expressed as:

$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}} \tag{5}$$

The implementation uses NumPy's built-in **np.tanh()** for efficiency.

Derivative: The derivative of the Tanh function, implemented **in line** 92, is computed as:

$$\tanh'(x) = 1 - \tanh^2(x) \tag{6}$$

This derivative is strongest near **x=0**, where it reaches a maximum of 1, and decays as **|x|** increases. This property aids in efficient training by providing significant gradients for values near zero.

2.3.2. Loss Functions

This section outlines the mathematical Representation and implementations of loss functions, including Mean Squared Error (MSE), Binary Cross Entropy, and Categorical Cross Entropy, along with their derivatives. These functions evaluate prediction accuracy and guide weight updates during training.

a) Mean Squared Error (MSE)

```
94  class Loss:
95      """Class containing static methods for loss functions and their derivatives."""
96      @staticmethod
97      def mse(y_true: np.ndarray, y_pred: np.ndarray) -> float:
98          """Mean Squared Error loss."""
99          return np.mean((y_true - y_pred) ** 2)
100     @staticmethod
101     def mse_derivative(y_true: np.ndarray, y_pred: np.ndarray) -> np.ndarray:
102         """MSE derivative."""
103         return 2 * (y_pred - y_true) / y_true.shape[0]
```

Mathematical Representation: implemented **in line** 99, calculates the average squared difference between predicted and true values. It is commonly used in regression tasks and is mathematically defined as:

$$\mathrm{MSE}(y_{\text{true}}, y_{\text{pred}}) = \frac{1}{N}\sum_{i=1}^{N}\left(y_{\text{true},i} - y_{\text{pred},i}\right)^2 \tag{7}$$

where **N** is the number of samples, **ytrue** are the ground truth labels, and **ypred** are the predicted outputs.

Derivative: The derivative of MSE, implemented **in line** 103, is used during backpropagation to compute gradients for weight updates. It is expressed as:

$$\frac{\partial \mathrm{MSE}}{\partial y_{\text{pred}}} = \frac{2}{N}\left(y_{\text{pred}} - y_{\text{true}}\right) \tag{8}$$

This gradient helps adjust the model's weights to reduce prediction error iteratively.

b) Binary Cross Entropy

```
107   @staticmethod
108   def binary_crossentropy(y_true: np.ndarray, y_pred: np.ndarray) -> float:
109       """Binary Cross Entropy loss."""
110       epsilon = 1e-15
111       y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
112       return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))
113   @staticmethod
114   def binary_crossentropy_derivative(y_true: np.ndarray, y_pred: np.ndarray) -> np.ndarray:
115       """Binary Cross Entropy derivative."""
116       epsilon = 1e-15
117       y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
118       return (y_pred - y_true) / (y_pred * (1 - y_pred) + epsilon)
```

Mathematical Representation: implemented **in line** 112, measures the performance of a binary classification model by comparing the true labels **ytrue** with the predicted probabilities **ypred**. A small constant (**ϵ =10−15)** ensures numerical stability by avoiding division or logarithm of zero. The BCE is mathematically defined as:

$$\mathrm{BCE}(y_{\text{true}}, y_{\text{pred}}) = -\frac{1}{N}\sum_{i=1}^{N}\left[y_{\text{true},i}\log\left(y_{\text{pred},i}\right) + \left(1 - y_{\text{true},i}\right)\log\left(1 - y_{\text{pred},i}\right)\right] \tag{9}$$

Derivative: The derivative of the BCE loss, implemented **in line** 118, is used for backpropagation to calculate gradients for weight updates. It is defined as:

$$\frac{\partial \mathrm{BCE}}{\partial y_{\text{pred}}} = \frac{y_{\text{pred}} - y_{\text{true}}}{y_{\text{pred}}\left(1 - y_{\text{pred}}\right) + \epsilon} \tag{10}$$

The predictions are clipped within a safe range using **np.clip** to prevent numerical instability during the calculation. This gradient helps adjust the model's weights to improve binary classification performance.

c) Categorical Cross Entropy

```
121  @staticmethod
122  def categorical_crossentropy(y_true: np.ndarray, y_pred: np.ndarray) -> float:
123      """Categorical Cross Entropy loss."""
124      epsilon = 1e-15
125      y_pred = np.clip(y_pred, epsilon, 1.0)
126      return -np.mean(np.sum(y_true * np.log(y_pred), axis=1))
127  @staticmethod
128  def categorical_crossentropy_derivative(y_true: np.ndarray, y_pred: np.ndarray) -> np.ndarray:
129      """Categorical Cross Entropy derivative."""
130      epsilon = 1e-15
131      y_pred = np.clip(y_pred, epsilon, 1.0)
132      return -y_true / (y_pred + epsilon)
```

Mathematical Representation: implemented **in line** 126, is used for multi-class classification. It evaluates the difference between the true one-hot encoded labels **ytruey** and the predicted probabilities **ypred** over **C** classes**. A** small constant **ϵ=10−15** ensures numerical stability by avoiding division or logarithm of zero. It is mathematically expressed as:

$$\mathrm{CCE}\left(y_{\text{true}}, y_{\text{pred}}\right) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{C} y_{\text{true},i,j} \log\left(y_{\text{pred},i,j}\right) \tag{11}$$

Derivative: The derivative of the CCE loss, implemented **in line** 132, is used for backpropagation to compute gradients for weight updates. It is defined as:

$$\frac{\partial \mathrm{CCE}}{\partial y_{\text{pred}}} = -\frac{y_{\text{true}}}{y_{\text{pred}} + \epsilon} \tag{12}$$

This gradient facilitates weight adjustments to minimize the error in multi-class predictions. The **np.clip** function is applied to maintain numerical stability during computation.

2.3.3. Softmax Function

The **softmax** function is used in multi-class classification problems. It converts raw scores (logits) into probabilities that sum up to 1 for each set of inputs.

```
38 def softmax(x: np.ndarray) -> np.ndarray:    try:
39     if x.ndim == 1:
40         x = x.reshape(1, -1)
41     shifted_x = x - np.max(x, axis=1, keepdims=True)
42     exp_x = np.exp(np.clip(shifted_x, -500, 500))
43     sum_exp = np.sum(exp_x, axis=1, keepdims=True)
44     result = exp_x / (sum_exp + 1e-10)
45     return result / np.sum(result, axis=1, keepdims=True)
46  except Exception as e:
47
48     print(f"Error in softmax calculation: {str(e)}")
49     return np.ones_like(x) / x.shape[1]
```

Mathematical Representation: The Softmax function, implemented **in line** 45, is used in multi-class classification to convert raw scores (logits) into normalized probabilities that sum to 1. It is mathematically expressed as:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}} \tag{13}$$

where **xi** represents the raw score for class **i**, and **n** is the total number of classes.

Implementation Details:

- **Input Handling**: Accepts a NumPy array. If the input is 1D, it is reshaped to a 2D array for compatibility with the computation.
- **Overflow Prevention**: Shifts input values by subtracting the maximum value in each row to prevent numerical overflow when computing exponentials.
- **Numerical Stability**: Clips values between [−500,500] and adds a small epsilon **10^-10** to the sum to avoid division by zero.
- **Output**: Returns a NumPy array where each row contains the probability distribution over classes for a sample.

2.4. Optimization Methods

The **update_weights** method which implements two optimization algorithms: Momentum and standard SGD (Stochastic Gradient Descent). **weight_gradients:** A list of gradients for the weights, computed during backpropagation. **bias_gradients:** A list of gradients for the biases, computed during backpropagation.

```
def update_weights(self, weight_gradients, bias_gradients):
    """Update weights based on selected optimizer"""
    if self.optimizer == "momentum":
        # Momentum update
        for i in range(len(self.weights)):
            self.v_weights[i] = self.momentum * self.v_weights[i] + \
                        self.learning_rate * weight_gradients[i]
            self.v_biases[i] = self.momentum * self.v_biases[i] + \
                        self.learning_rate * bias_gradients[i]
            self.weights[i] -= self.v_weights[i]
            self.biases[i] -= self.v_biases[i]
    else:
        # Standard SGD update
        for i in range(len(self.weights)):
            self.weights[i] -= self.learning_rate *weight_gradients[i]
                        self.biases[i] -= self.learning_rate * bias_gradients[i]
```

a) Momentum-Based Optimization

Combines the momentum term and the gradient to compute updates. Momentum helps smooth the optimization trajectory and accelerates convergence. Updates are computed as:

$$\begin{aligned} v_{\text{weights}}^{(t)} &= \text{momentum} \cdot v_{\text{weights}}^{(t-1)} + \text{learning rate} \cdot \text{weight gradients} \\ v_{\text{biases}}^{(t)} &= \text{momentum} \cdot v_{\text{biases}}^{(t-1)} + \text{learning rate} \cdot \text{bias gradients} \end{aligned} \tag{14}$$

The weights and biases are updated using:

$$\begin{aligned} \text{weights} &= \text{weights} - v_{\text{weights}} \\ \text{biases} &= \text{biases} - v_{\text{biases}} \end{aligned} \tag{15}$$

Loops through each layer of the network:

1. Updates the velocity terms **(v_weights, v_biases)** using momentum and gradients.
2. Updates the weights and biases by subtracting the velocity terms.

b) Standard Stochastic Gradient Descent (SGD)

Directly adjusts weights and biases using gradients:

$$\begin{aligned} \text{weights} &= \text{weights} - \text{learning rate} \cdot \text{weight gradients} \\ \text{biases} &= \text{biases} - \text{learning rate} \cdot \text{bias gradients} \end{aligned} \tag{16}$$

Loops through each layer of the network:

1. Updates the weights and biases using their respective gradients and the learning rate.

2.5. Data Processing Pipeline

This section details the system's procedures for data collection, validation, normalization, and preparation, ensuring the dataset is efficiently structured and standardized for optimal neural network training.

2.5.1. Data Collection

The system implements an interactive data collection mechanism through mouse clicks on a 2D plot: In the **on_click method** handles User click event capture, Data point validation, Dynamic array management for points and labels, Real-time visualization updates.

```
def on_click(self, event):
    """Handle mouse click event for adding data points"""
    if event.inaxes != self.ax:
        return
    try:
        current_label = int(self.current_label.get())
        num_classes = int(self.class_var.get()
        if current_label > num_classes:
            messagebox.showwarning("Invalid Label",
                f"Label {current_label} cannot be greater than number of classes {num_classes}")
            return
        # Add new point
        x = np.array([event.xdata, event.ydata])
        self.sample_count += 1
        # Update data arrays
```

```
1338        if len(self.points) == 0:
1339            self.points = np.array([x])
1340            self.labels = np.array([current_label])
1341        else:
1342            self.points = np.vstack([self.points, x])
1343            self.labels = np.append(self.labels, current_label)
```

2.5.2. Data Preprocessing

Before training, data undergoes preprocessing:

```
1393    def normalize_data(self):
1394        """Normalize training data"""
1395        try:
1396            # Compute mean and standard deviation
1397            self.mean = np.mean(self.points, axis=0)
1398            self.std = np.std(self.points, axis=0)
1399            self.std[self.std == 0] = 1  # Avoid division by zero
1400            # Normalize points
1401            normalized_points = (self.points - self.mean) / self.std
1402            # Create one-hot encoded labels
1403            num_classes = int(self.class_var.get())
1404            one_hot_labels = np.zeros((len(self.labels), num_classes))
1405            for i, label in enumerate(self.labels):
1406                one_hot_labels[i, int(label)-1] = 1
1407            return normalized_points, one_hot_labels
1408        except Exception as e:
1409            messagebox.showerror("Normalization Error", str(e))
1410            return None, None
```

The **normalize_data** function prepares dataset features and labels by standardizing numerical features (**self.points**: A NumPy array containing the dataset). and applying one-hot encoding to categorical labels **(self.labels:** A NumPy array containing the corresponding labels for the dataset).

Data Normalization: The function calculates the mean **(self.mean)** and standard deviation **(self.std)** for each feature, replacing zero standard deviations with 1 to avoid division errors, and rescales the data to have a mean of 0 and a standard deviation of 1 in **(self.std).** implemented **in line** 1401, is computed as:

$$\text{normalized_points} = \frac{\text{self.points} - \text{self.mean}}{\text{self.std}} \tag{17}$$

One-Hot Encoding: Labels are transformed into a one-hot encoded matrix by determining the number of unique classes are determined from **self.class_var.get()** (likely a method or property to fetch the number of classes), initializing a zero matrix, and assigning a 1 to the appropriate position for each label.

The **normalize_data** function produces normalized feature data **(normalized_points)** and a one-hot encoded label matrix **(one_hot_labels).**

2.6. Neural Network Implementation

This section focuses on the design, initialization, and operation of a single-layer neural network, covering its architecture, forward propagation, backward propagation, optimization strategies, training process, and evaluation methods. It emphasizes efficient learning and accurate predictions specifically tailored to tasks suitable for single-layer networks.

2.6.1. Network class Initialization

The **__init__ method** initializes various parameters of the neural network, validates configurations, and sets up the network's attributes.

```
class NeuralNetwork:
    def __init__(self, input_size: int,
                 hidden_sizes: Union[List[int], int, None],
                 output_size: int,
                 training_method: str = "Single layer",
                 hidden_activations: Union[List[str], str] = "sigmoid",
                 output_activation: str = "sigmoid",
                 use_softmax: bool = False,
                 loss_function: str = "mse",
                 learning_rate: float = 0.03,
                 optimizer: str = "momentum",
                 momentum: float = 0.95,
                 l2_lambda: float = 0.0):
        self.is_multi = (training_method == "Multi layer")
        self.input_size = input_size
        self.output_size = output_size
        self.hidden_sizes = hidden_sizes if isinstance(hidden_sizes, list) else []
```

```python
        self.learning_rate = learning_rate
        self.initial_learning_rate = learning_rate
        self.l2_lambda = l2_lambda
        self.use_softmax = use_softmax
        self.loss_function = loss_function
        self.optimizer = optimizer
        self.momentum = momentum
        # Activation function mapping
        self.activation_functions = {
            'sigmoid': (Activation.sigmoid, Activation.sigmoid_derivative),
            'relu': (Activation.relu, Activation.relu_derivative),
            'tanh': (Activation.tanh, Activation.tanh_derivative)
        }
        # Convert single activation to list for hidden layers
        if isinstance(hidden_activations, str):
            hidden_activations = [hidden_activations]
        self.hidden_activations = hidden_activations
        self.output_activation = output_activation
        # Validate hidden layer configuration
        if self.is_multi:
            if not hidden_sizes or not 1 <= len(hidden_sizes) <= 3:
                raise ValueError("Multi-layer network must have 1-3 hidden layers")
            if any(size < 1 for size in hidden_sizes):
                raise ValueError("Hidden layer sizes must be positive")
            if len(hidden_activations) != len(hidden_sizes):
                raise ValueError("Number of activation functions must match number of hidden layers")
        # Initialize network
        self.initialize_network()
        # Initialize training history
        self.error_history = []
        self.validation_error_history = []
```

a) Parameters

1. **input_size (int):** The number of input features in the dataset.
2. **hidden_sizes (list of ints, int, or None):** for single layer it will be None.
3. **output_size (int):** The number of output neurons (typically equal to the number of output classes or regression targets).

4. **training_method (str):”** Single layer" or "multi-layer", specifying the type of network.
5. **hidden_activations (list of str or str):** none
6. **output_activation (str):** The activation function used for the output layer (default: "sigmoid").
7. **use_softmax (bool):** If True, applies a **softmax** function at the output layer (commonly for classification problems).
8. **loss_function (str):** The type of loss function (e.g., "mse" for mean squared error).
9. **learning_rate (float):** Learning rate for training (default: 0.03).
10. **optimizer (str):** Optimizer type (e.g., "momentum").
11. **momentum (float):** Momentum parameter for the optimizer (default: 0.95).
12. **l2_lambda (float):** L2 regularization parameter (default: 0.0).

b) Attributes

- **self.is_multi**: Determines whether the network is multi-layer based on **training_method.**
- **self.activation_functions**: A dictionary mapping activation function names to their corresponding function and derivative (e.g., sigmoid and sigmoid derivative).
- **self.error_history** and **self.validation_error_history**: Store training and validation error histories.

2.6.2. Network Method Initialization

The **initialize_network** method sets up the weights, biases, and optionally momentum variables for the neural network**.**

```
def initialize_network(self) -> None:
    self.weights = []
    self.biases = []
    # Only initialize momentum variables if using momentum optimizer
    if self.optimizer == "momentum":
        self.v_weights = []
        self.v_biases = []
    if self.is_multi:
```

```
        layer_sizes = [self.input_size] + self.hidden_sizes + [self.output_size]
        for i in range(len(layer_sizes) - 1):
            scale = np.sqrt(2.0 / (layer_sizes[i] + layer_sizes[i+1]))
            weights = np.random.normal(0, scale, (layer_sizes[i+1], layer_sizes[i]))
            biases = np.zeros(layer_sizes[i+1])
            self.weights.append(weights)
            self.biases.append(biases)
            if self.optimizer == "momentum":
                self.v_weights.append(np.zeros_like(weights))
                self.v_biases.append(np.zeros_like(biases))
    else:
        scale = np.sqrt(2.0 / (self.input_size + self.output_size))
        weights = np.random.normal(0, scale, (self.output_size, self.input_size))
        biases = np.zeros(self.output_size)
        self.weights = [weights]
        self.biases = [biases]
        if self.optimizer == "momentum":
            self.v_weights = [np.zeros_like(weights)]
            self.v_biases = [np.zeros_like(biases)]
    self.store_best_weights()
```

For single-layer networks: weights and biases are directly initialized between the input and output layers.

Momentum Variables: If the optimizer is "momentum", initializes additional variables **(v_weights and v_biases)** to track momentum updates for each weight and bias.

Best Weights Storage: **store_best_weights** method (not shown): Presumably saves the current weights and biases as the "best" configuration, likely for use in early stopping or rollback after validation.

2.6.3. Forward Propagation

The forward method calculates the output of the network for a given input x by propagating it through the layers of the network. It supports both single-layer and multi-layer configurations.

```python
    def forward(self, x: np.ndarray) -> Union[np.ndarray, Tuple[np.ndarray, List[np.ndarray], List[np.ndarray]]]:
        try:
            if x.ndim == 1:
                x = x.reshape(1, -1)
            if self.is_multi:
                layer_outputs = []
                layer_inputs = []
                current_input = x
                # Process hidden layers
                for i in range(len(self.weights) - 1):
                    net = np.dot(current_input, self.weights[i].T) + self.biases[i]
                    layer_inputs.append(net)
                    activation_func = self.activation_functions[self.hidden_activations[i]][0]
                    current_input = activation_func(net)
                    layer_outputs.append(current_input)
                # Output layer with softmax option
                final_net = np.dot(current_input, self.weights[-1].T) + self.biases[-1]
                layer_inputs.append(final_net)
                if self.use_softmax:
                    final_output = softmax(final_net)  # Use softmax function
                else:
                    output_func = self.activation_functions[self.output_activation][0]
                    final_output = output_func(final_net)
                return final_output, layer_outputs, layer_inputs
            else:
                # Single layer forward pass
                net = np.dot(x, self.weights[0].T) + self.biases[0]
                activation_func = self.activation_functions[self.output_activation][0]
                return activation_func(net), None, [net]
        except Exception as e:
            raise RuntimeError(f"Error in forward pass: {str(e)}")
```

a) **Input: x (np.ndarray):** The input data for the forward pass**. If x.ndim == 1** (a single example), it reshapes x to ensure compatibility with the network's weight matrix.

b) **Returns: For single-layer networks:** Output of the single layer and pre-activation values.

c) **Workflow of Single-Layer Networks configurations:** implemented **in line** 287

Computes the pre-activation (net) as:

$$\text{net} = x \cdot \text{weights}^T + \text{biases} \tag{18}$$

And applies the activation function specified for the output layer. Then Returns the output and pre-activation value (net).

Applies the activation function: If **use_softmax** is True, applies the softmax function to **final_net**. Otherwise, applies the activation function specified for the output layer.

2.6.4. Backward Propagation

The backward method computes gradients for the network weights and biases using backpropagation, updates the parameters with the chosen optimizer, and returns the computed loss for the batch after the backward pass.

```
321 def backward(self, X: np.ndarray, y: np.ndarray,
322         outputs: Union[np.ndarray, Tuple[np.ndarray, List[np.ndarray], List[np.ndarray]]]) -> float:
323     """Modified backward pass to use the selected optimizer"""
324     batch_size = len(X)
325     try:
326         if self.is_multi:
327             final_output, layer_outputs, layer_inputs = outputs
328             layer_inputs_with_x = [X] + layer_outputs
329             # Calculate initial error based on loss function and softmax use
330             if self.use_softmax:
331                 # For softmax + categorical crossentropy, the derivative simplifies
332                 delta = final_output - y
333             else:
334                 # For other combinations, apply appropriate derivatives
335                 if self.loss_function == "mse":
336                     delta = Loss.mse_derivative(y, final_output)
337                 elif self.loss_function == "binary_crossentropy":
338                     delta = Loss.binary_crossentropy_derivative(y, final_output)
339                 elif self.loss_function == "categorical_crossentropy":
340                     delta = Loss.categorical_crossentropy_derivative(y, final_output)
341                 # Apply activation derivative if not using softmax
342                 output_derivative = self.activation_functions[self.output_activation][1]
```

```
            delta *= output_derivative(layer_inputs[-1])
        # Store output layer gradients
        weight_gradients = [np.zeros_like(w) for w in self.weights]
        bias_gradients = [np.zeros_like(b) for b in self.biases]
        weight_gradients[-1] = np.dot(delta.T, layer_outputs[-1]) / batch_size
        bias_gradients[-1] = np.mean(delta, axis=0)
        # Backpropagate through hidden layers
        for i in range(len(self.weights) - 2, -1, -1):
            error = np.dot(delta, self.weights[i + 1])
            activation_derivative = self.activation_functions[self.hidden_activations[i]][1]
            delta = error * activation_derivative(layer_inputs[i])
            weight_gradients[i] = np.dot(delta.T, layer_inputs_with_x[i]) / batch_size
            bias_gradients[i] = np.mean(delta, axis=0)
    else:  # Single layer
        final_output, _, layer_inputs = outputs
        # Calculate error based on loss function
        if self.loss_function == "mse":
            delta = Loss.mse_derivative(y, final_output)
        elif self.loss_function == "binary_crossentropy":
            delta = Loss.binary_crossentropy_derivative(y, final_output)
        elif self.loss_function == "categorical_crossentropy":
            if self.use_softmax:
                delta = final_output - y
            else:
                delta = Loss.categorical_crossentropy_derivative(y, final_output)
        # Apply activation derivative if not using softmax
        if not self.use_softmax:
            activation_derivative = self.activation_functions[self.output_activation][1]
            delta *= activation_derivative(layer_inputs[0])
        # Calculate gradients
        weight_gradients = [np.dot(delta.T, X) / batch_size]
        bias_gradients = [np.mean(delta, axis=0)]
    # Add L2 regularization gradients if needed
    if self.l2_lambda > 0:
        for i in range(len(self.weights)):
            weight_gradients[i] += (self.l2_lambda / batch_size) * self.weights[i]
    # Update weights using the selected optimizer
    self.update_weights(weight_gradients, bias_gradients)
    # Compute and return loss
    return self.compute_loss(y, final_output)
```

```
383     except Exception as e:
384         print(f"Error in backward pass: {str(e)}")
385         return float('inf')
```

a) Inputs

- **X:** Input data for the current batch (numpy array of shape (batch_size, features)).
- **y:** True labels corresponding to X (numpy array of shape (batch_size, output_size)).

b) outputs: final_output: The final output from the network (predicted values).

c) attributes:

- **self.is_multi:** A boolean indicating if the network has multiple layers.
- **self.loss_function:** The chosen loss function (e.g., "mse", "binary_crossentropy", or "categorical_crossentropy").
- **self.use_softmax:** Indicates if the output layer uses softmax activation.
- **self.weights & self.biases:** Network weights and biases for each layer.
- **self.l2_lambda:** Regularization coefficient for L2 regularization.
- **self.activation_functions:** Dictionary mapping activation names to their functions and derivatives.
- **self.update_weights:** Method to update weights using the optimizer.

Backpropagation Through Layers

For single-layer networks: Compute gradients directly using the input X and delta.

L2 Regularization: implemented in line 375 If **self.l2_lambda > 0**, gradients for weights are adjusted to include the L2 regularization term:

$$\text{weight_gradients}\,[i] \mathrel{+}= \frac{\lambda}{\text{batch_size}} \cdot \text{weights}\,[i] \tag{19}$$

2.6.5. Training Process

The **train** function trains the model on input data X and target labels y. Optionally validates the model using **validation_data**. Implements early stopping based on validation error and target error. Adjusts the learning rate adaptively during training. Tracks the error history during training and validation.

```python
def train(self, X: np.ndarray, y: np.ndarray,
        validation_data: Optional[Tuple[np.ndarray, np.ndarray]] = None,
        max_epochs: int = 1000,
        target_error: float = 0.01,
        batch_size: int = 32,
        patience: int = 10,
        min_learning_rate: float = 0.0001) -> Tuple[int, float]:
    num_samples = len(X)
    self.error_history = []
    self.validation_error_history = []
    best_error = float('inf')
    best_epoch = 0
    patience_counter = 0
```

a) Inputs

- **X:** Training data (numpy array of shape (num_samples, features)).
- **y:** Target labels for training data.
- **validation_data:** Optional tuple (X_val, y_val) for validation monitoring.
- **max_epochs:** Maximum number of training epochs (default: 1000).
- **target_error:** Error threshold to stop training early (default: 0.01).
- **batch_size:** Number of samples per mini-batch (default: 32).
- **patience:** Number of epochs to wait for validation improvement before stopping (default: 10).
- **min_learning_rate:** Minimum allowable learning rate (default: 0.0001).

b) Initialization

- **error_history:** List of training errors for each epoch.
- **validation_error_history:** List of validation errors (if provided).
- **best_error and best_epoch:** Tracks the lowest validation error and its epoch.
- **patience_counter:** Counts epochs with no improvement in validation error.

c) Training Loop Initialization: The training loop iterates over **max_epochs** or until early stopping criteria are met. Implements data shuffling for each epoch. Shuffle data indices for randomness.

```
try:
    # Training loop
    for epoch in range(max_epochs):
        # Shuffle training data
        indices = np.random.permutation(num_samples)
        X_shuffled = X[indices]
        y_shuffled = y[indices]
```

d) Mini-Batch Processing: Divides data into mini-batches **using batch_size** and processes each batch separately and Accumulates error for monitoring.

```
        total_error = 0
        num_batches = (num_samples + batch_size - 1) // batch_size
        for batch in range(num_batches):
            start_idx = batch * batch_size
            end_idx = min(start_idx + batch_size, num_samples)
            X_batch = X_shuffled[start_idx:end_idx]
            y_batch = y_shuffled[start_idx:end_idx]
```

e) Forward and Backward Passes: For each mini-batch Perform a forward pass using **self.forward**. Compute gradients and update weights via the **self.backward** method. Accumulate batch errors.

```
            # Forward and backward passes
            outputs = self.forward(X_batch)
            batch_error = self.backward(X_batch, y_batch, outputs)
            total_error += batch_error
```

f) Error Monitoring and Early Stopping: Tracks training and validation errors and Updates best model if improvement seen and Manages patience counter for early stopping. Average error across all mini-batches is calculated for the epoch:

$$\text{avg_error} = \frac{\text{total_error}}{\text{num_batches}} \tag{20}$$

The **avg_error** is stored in **self.error_history.** If **validation_data** is provided, the model computes the validation error using **self.compute_loss:**

$$\text{val_error} = \text{self.compute_loss}\ (y_{\text{val}}, \text{val_outputs}) \tag{21}$$

The current error for evaluation is set to **val_error** (if validation data exists) or **avg_error** otherwise.

For Early Stopping If the current error improves on **best_error**: Update **best_error** and **best_epoch**. Save the best weights using **self.store_best_weights**. Reset the **patience_counter.** Otherwise, increment the **patience_counter**. If **patience_counter** exceeds patience, early stopping is triggered.

```
# Compute validation error if validation data provided
if validation_data is not None:
    X_val, y_val = validation_data
    val_outputs = self.forward(X_val)
    val_error = self.compute_loss(y_val,
                                  val_outputs[0] if self.is_multi else val_outputs)
    self.validation_error_history.append(val_error)
    current_error = val_error
else:
    current_error = avg_error

# Update best error and weights
if current_error < best_error:
    best_error = current_error
    best_epoch = epoch + 1
    self.store_best_weights()
    patience_counter = 0
else:
    patience_counter += 1
```

g) Adaptive Learning Rate: Adjusts learning rate based on error trends and Prevents learning rate from becoming too small and Allows recovery from poor learning situations.

- If training error increases compared to the previous epoch:

$$\text{learning_rate} = \max\ (\text{learning_rate} \times 0.5, \text{min_learning_rate}) \tag{22}$$

- If error decreases, the learning rate is increased slightly:

$$\text{learning_rate} = \min\ (\text{learning_rate} \times 1.1, \text{initial_learning_rate}) \tag{23}$$

```
# Adaptive learning rate
if epoch > 0 and self.error_history[-1] > self.error_history[-2]:
    self.learning_rate = max(self.learning_rate * 0.5, min_learning_rate)
```

```
        else:
            self.learning_rate = min(self.learning_rate * 1.1, self.initial_learning_rate)
```

h) Progress Reporting: Every 100 epochs, the average error is printed to monitor training progress. Messages are displayed for early stopping or when the target error is reached.

```
        # Print progress every 100 epochs
        if (epoch + 1) % 100 == 0:
            print(f"Epoch {epoch + 1}/{max_epochs}, Error: {avg_error:.6f}")
        # Early stopping checks
        if patience_counter >= patience:
            print(f"Early stopping at epoch {epoch + 1}")
            break
        if avg_error <= target_error:
            print(f"Reached target error at epoch {epoch + 1}")
            break
```

i) Restore Best Weights and return: After training, the best weights (from the epoch with the lowest error) are restored using **self.restore_best_weights**. And return **best_epoch:** The epoch where the best validation (or training) error occurred. **best_error:** The lowest validation (or training) error achieved.

```
        # Restore best weights
        self.restore_best_weights()
        return best_epoch, best_error
```

2.6.6. Prediction

The predict method generates predictions for a given input dataset X by Performing a forward pass through the network to obtain outputs and Converts network outputs to class predictions by electing the most probable class index for each input sample (using argmax). Returns a 1D numpy array of predicted class indices for the input dataset X

```
    def predict(self, X: np.ndarray) -> np.ndarray:
        try:
            outputs = self.forward(X)
            final_output = outputs[0] if self.is_multi else outputs
            return np.argmax(final_output, axis=1)
        except Exception as e:
            warnings.warn(f"Error making predictions: {str(e)}")
            return np.zeros(len(X))
```

a) Inputs

- **X:** A numpy array representing the input data (shape: (batch_size, features)).
- **self.is_multi**: A boolean indicating if the network has multiple layers.

B) Determine Final Output: If the network is multi-layer (self.is_multi), outputs is assumed to be a tuple or list, and the first element (outputs[0]) contains the final layer's predictions. Otherwise, outputs itself is treated as the final output.

D) Class Prediction: The argmax function is applied along **the last axis (axis=1)** to extract the index of the highest probability for each sample in the batch. The result is a 1D array of class indices corresponding to the predicted class for each input sample.

2.6.7. Loss Computation

The function computes a loss value based on a specified loss function (mse, binary_crossentropy, or categorical_crossentropy). Additionally, it can include an L2 regularization term if specified. Returns the computed loss, which includes the base loss (selected function) and optionally the L2 regularization term.

```
def compute_loss(self, y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """Compute loss based on selected loss function"""
    batch_size = len(y_true)
    try:
        # Get loss based on selected function
        if self.loss_function == "mse":
            loss = Loss.mse(y_true, y_pred)
        elif self.loss_function == "binary_crossentropy":
            loss = Loss.binary_crossentropy(y_true, y_pred)
        elif self.loss_function == "categorical_crossentropy":
            loss = Loss.categorical_crossentropy(y_true, y_pred)
        else:
            raise ValueError(f"Unknown loss function: {self.loss_function}")
        # Add L2 regularization term if needed
        if self.l2_lambda > 0:
            l2_term = 0
            for weights in self.weights:
                l2_term += np.sum(weights ** 2)
            loss += (self.l2_lambda / (2 * batch_size)) * l2_term
        return loss
```

a) Inputs

- **y_true (numpy array):** The true labels.
- **y_pred (numpy array):** The predicted outputs.
- **self.loss_function:** Specifies which loss function to use.
- **self.l2_lambda**: Coefficient for L2 regularization.
- **self.weights:** Model weights (used for L2 regularization).

b) Batch Size Calculation:

The size of the batch (number of samples) is calculated from **y_true.**

c) Loss Function Selection:

The loss function is chosen based on the value of self.loss_function

d) L2 Regularization (Optional):

If **self.l2_lambda** is greater than **0**, L2 regularization is applied. The L2 regularization term is added to the loss and computed as:

$$\text{L2_term} = \frac{\lambda}{2 \cdot \text{batch_size}} \cdot \sum\left(\text{weights}^2\right) \tag{24}$$

3. Results and Analysis

This section presents the outcomes of training single neural network, including performance metrics, visualizations of decision boundaries, error convergence analysis, and the network's ability to classify data accurately for both binary and multi-class scenarios.

3.1. Single Neuron Results

This subsection highlights the performance of a single neuron network trained for binary classification.

3.1.1. Training Performance

Neural network trained using Binary Cross Entropy loss and momentum optimization (learning rate=0.3, momentum=0.98). Training converged rapidly in 17 epochs, achieving a final error of 0.000049, demonstrating efficient learning behavior.

3.1.2. Decision Boundaries

Dataset comprised 41 points in 2D space, with clear linear separation between classes (blue/orange). Decision boundaries visualized through both linear boundaries and colored regions.

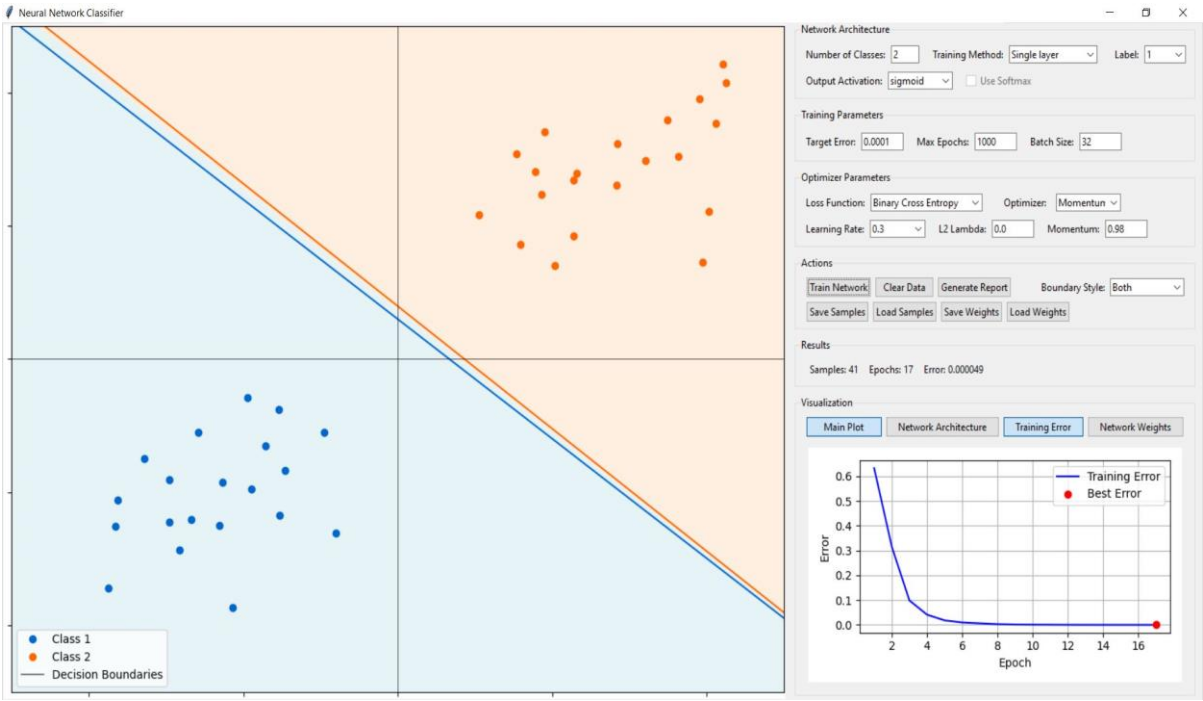

Figure 4. Decision Boundaries of two classes classification

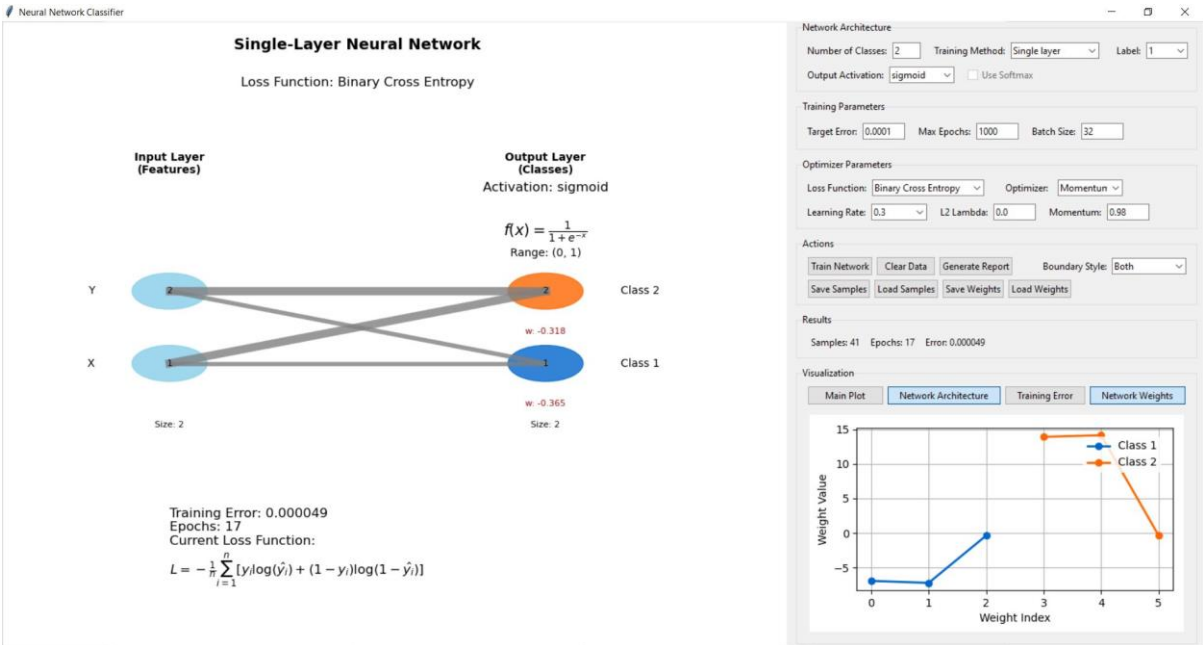

Figure 5. Network Architecture of two classes classification

Network architecture shows 2-2 configuration with sigmoid activation and final weights of -0.318 and -0.365.

3.1.3. Performance Analysis

Network achieved perfect classification with no misclassified points. Error curve shows stable, rapid convergence without overfitting. Low final error (0.000049) and optimal decision boundary placement indicate excellent generalization capability. Quick convergence time validates the chosen architecture and hyperparameters for this binary classification task.

3.2. Multi Neuron Results

This subsection details the performance of a multi-neuron network for multi-class classification.

3.2.1. Training Performance

Network trained using MSE loss and momentum optimization (learning rate=1.0, momentum=0.98). Training completed in 116 epochs with final error of 0.007548.

3.2.2. Decision Boundaries

Dataset: 52 samples distributed across 5 classes (blue, orange, green, purple, yellow) . Clear linear separation between all classes.

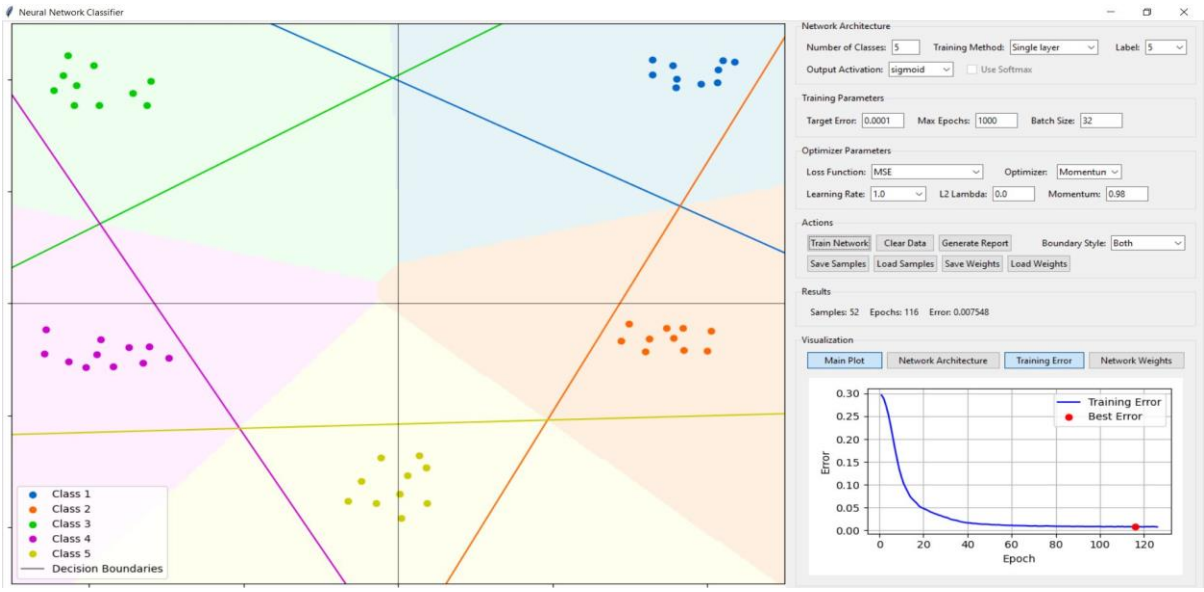

Figure 6. Decision Boundaries of Multiple classes classification

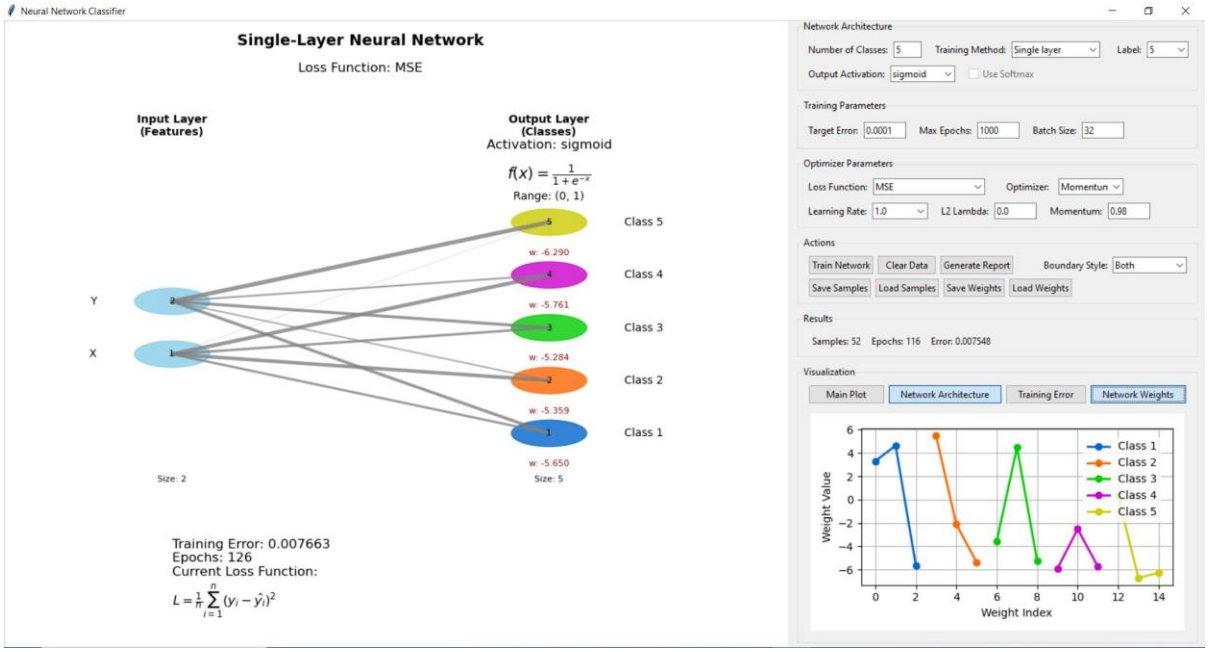

Figure 7. Network Architecture of Multiple classes classification

Network architecture shows 2-5 configuration (2 inputs: X, Y; 5 outputs: one per class) . Final weights range from -6.290 to -5.284, indicating effective class boundaries**.**

3.2.3. Performance Analysis

Successfully separated 5 classes with no misclassified points .Decision boundaries form well-defined regions for each class .MSE error of 0.007548 indicates strong classification accuracy .Network's ability to distinguish 5 classes demonstrates successful multi-class classification capability .This implementation effectively demonstrates the network's ability to handle multiple classes with linear decision boundaries.

4. Conclusions

Successfully implemented both binary and multi-class classification**.** Demonstrated effective learning with momentum optimization**.** Achieved perfect classification in both scenarios**.** Real-time visualization provides intuitive understanding.